- **Cache types:** there are also different ways to plug in a cache into a system: we have seen the difference between write-ahead, read-ahead, write-through, write-behind, refresh-ahead. This choice may be as important, or usually even more, than choosing the type of cache to employ.
- **Thread-safety:** when we share a resource (in this case a cache) between multiple threads, we need to be very careful in designing it so that we won't end up with race conditions, inconsistent data, or even worse deadlocks.
- If we use **functional programming**, we might make our life easier when it comes to concurrency.
- **Consistent hashing**: a new type of hashing we hadn't presented yet, consistent hashing helps us when we have a hash table whose size may vary in time dynamically, and we don't get the chance of recreating the table from scratch every time.

---

[1] Graphics Processing Unit: originally designed to speed up image processing and buffering for display devices, with the turn of the century they become increasingly used as general (parallel) computation devices, so much so they actually are the election choice, instead of CPUs (Central Processing Unit), for algebraic-intensive tasks like machine learning.

[2] Cursors are control structures that allow traversing a set of rows in a DB table; as a simplification, they can be thought of pointers to the next row to read in a portion of the table, and they are used to subsequently process rows (either reading data or modifying it) one by one, as opposite to a batch read/write where a group or records is read/written at once.

[3] Locks are another DB constructs that prevent all reads/writes on a raw/table/DB while data is being modified. They are needed for consistency, but they can have a dramatic impact on availability. In general, using cursors that allow updating the data is not advised because they might hold locks for a long time.

[4] Not literally! Just jargon.